

L&T EduTech

Deep Learning from Production to Deployment

Task 6: Creating a Streamlit User Interface

Technical Project Report

Field	Detail
Project Title	CIFAR-10 Image Classifier — Streamlit Web Application
Student Name	Sharon Varghese Thoppil
Course / Program	L&T EduTech — Deep Learning from Production to Deployment
Roll Number / Student ID	41 (5621471)
Streamlit app	https://sharonvarghesethoppil-l-t-task-6-app-bc6t1c.streamlit.app/
Github	https://github.com/SharonVargheseThoppil/L-T-Task-6

Table of Contents

Contents

Table of Contents	2
1. Introduction	4
2. Objective	4
3. Task Requirements	4
4. Reference Model and Dataset	5
5. Technologies and Tools Used	6
6. System Requirements	6
6.1 Software Requirements	6
6.2 Hardware Requirements	6
7. Project Folder Structure	7
8. Application Architecture	7
9. Streamlit User Interface Design	7
10. User Input and Widgets	8
11. Model Integration	9
12. Image / Data Preprocessing	9
13. Prediction Process	9
14. Error Handling	10
15. User Interface Screenshots	10
16. Image Upload Screenshots	17
17. Prediction Output Screenshots	17
18. Prediction Probability Screenshots	17
Dog image (Figures 5–6)	17
Airplane image (Figures 9–10)	17
Horse image (Figures 12–13)	18
19. Results	18
20. Functional Evaluation	18
21. User Experience and Interface Design	19
22. Observations	19
23. Limitations	20
24. Future Enhancements	20
25. Conclusion	21
26. References	21
27. Appendix	21
Appendix A — Source Code (app.py)	21

Appendix B — Requirements File.....	29
Appendix C — Additional Screenshots.....	29

1. Introduction

This report documents Task 6 of the L&T EduTech Deep Learning program, titled “Creating a Streamlit User Interface.” The task requires building a web-based interface that allows a user to interact with a previously trained deep learning model without needing to write or run any code directly. Streamlit is a Python framework that converts a plain Python script into an interactive web application, using simple function calls to create titles, file uploaders, buttons, sidebars, and result displays. It is widely used for rapidly building demonstration interfaces for machine learning and deep learning models because it removes the need for separate front-end web development (HTML, CSS, or JavaScript).

For this task, the trained model from an earlier task — a Convolutional Neural Network (CNN) built on the CIFAR-10 dataset — was connected to a Streamlit interface named `app.py`. The resulting application, referred to on-screen as the “CIFAR-10 Image Classifier,” allows a user to upload an image, view basic information about it, trigger a prediction, and see the predicted class along with a confidence score and a breakdown of the top prediction probabilities.

The observations, screenshots, and code excerpts presented in this report are based strictly on the material submitted for review: thirteen screenshots covering the project folder structure, the application source code (as visible in an editor window), and three end-to-end usage runs of the deployed application. No project ZIP file, complete `app.py` file, or `requirements.txt` file was submitted alongside the screenshots; accordingly, wherever a claim cannot be verified from the submitted screenshots, this report explicitly states “Not provided in the submitted project materials” rather than assuming or inventing details.

2. Objective

Official Task Objective: *To build a user-friendly web interface for interacting with deep learning prediction models.*

The submitted implementation addresses this objective by presenting the previously trained CIFAR-10 CNN model through a Streamlit web page. As demonstrated in the screenshots, a user can upload an image file through a simple upload widget, view the uploaded image and basic file information, and click a single “Predict Image” button to obtain the model's prediction. The result is displayed as a named class (e.g., DOG, AIRPLANE, HORSE), a percentage confidence value, a visual confidence bar, and a ranked list of the top five class probabilities. This removes the need for the end user to interact with Python code, a Jupyter Notebook, or the command line in order to test the model, which is consistent with the stated objective of the task.

3. Task Requirements

The official requirements for Task 6, as specified by L&T EduTech, are as follows:

- Streamlit UI design
- User input forms/widgets
- Model prediction integration

- Prediction output display
- Usability and layout improvements

Sections 10 through 16 of this report examine each of these requirements against the evidence found in the submitted screenshots and source-code excerpt.

4. Reference Model and Dataset

Attribute	Verified Value
Model filename	cifar10_cnn_model.keras
Dataset	CIFAR-10
Model type	Convolutional Neural Network (CNN)
Framework	TensorFlow / Keras
Interface	Streamlit
Model input size	32 × 32 pixels
Number of output classes	10

The sidebar of the application lists the ten classes supported by the model under the heading “Supported Classes.” These correspond exactly to the standard CIFAR-10 class set:

- Airplane
- Automobile
- Bird
- Cat
- Deer
- Dog
- Frog
- Horse
- Ship
- Truck

The CIFAR-10 dataset is a well-known benchmark dataset consisting of 60,000 32×32 color images distributed across these 10 classes, commonly used for teaching and evaluating image classification models. Its use here is confirmed both by the sidebar label “Dataset: CIFAR-10” and by the model's declared input size of 32×32 pixels, which matches the native CIFAR-10 image resolution.

5. Technologies and Tools Used

The following technologies were directly confirmed from the visible source code, the application's own on-screen “Model Information” panel, or the VS Code editor tabs/terminal shown in Figure 1:

Technology	Purpose	Evidence
Python	Core application language (app.py)	File extension and VS Code editor (Fig. 1)
Streamlit	Web interface framework; used to run the app and render widgets	Sidebar label “Interface: Streamlit”; terminal command “streamlit run app.py” (Fig. 1)
TensorFlow / Keras	Model loading and inference (tf.keras.models.load_model)	Sidebar label “Framework: TensorFlow / Keras”; source code (Fig. 1)
os module (Python standard library)	Building the model file path and checking file existence	Source code: os.path.join, os.path.dirname, os.path.exists (Fig. 1)

6. System Requirements

6.1 Software Requirements

Requirement	Verified Value
Python version	Not provided in the submitted project materials
Streamlit version	Not provided in the submitted project materials
TensorFlow / Keras version	Not provided in the submitted project materials
Other dependencies	Not provided in the submitted project materials — requirements.txt exists in the project folder (Figure 1) but its contents were not included in the submitted screenshots

6.2 Hardware Requirements

No specific hardware requirements are documented in the submitted materials. The application was run and demonstrated locally (see Section 9), which only requires a standard development machine capable of running Python, TensorFlow, and a local web browser; no GPU or specialized hardware usage is claimed or evidenced.

7. Project Folder Structure

The following folder structure is reproduced exactly as shown in the VS Code Explorer panel in Figure 1:

```
Task6_Streamlit_CIFAR10/
|
+-- venv/
+-- app.py
+-- cifar10_cnn_model.keras
+-- README.md
+-- requirements.txt
```

The venv/ folder is a local Python virtual environment used to isolate the project's dependencies (visible as an active environment in the VS Code terminal prompt: “(venv) PS D:\Sharon\Task6_Streamlit_CIFAR10>”). Virtual environment folders are conventionally excluded from source-control submissions because they contain machine-specific, regenerable package installations rather than project source files; whether this project excludes venv/ from any version-control or submission package was not confirmed, as no .gitignore file or repository listing was included in the submitted materials.

8. Application Architecture

Based on the observed behaviour of the application across the three demonstrated runs (Figures 2–13), the application follows this flow:

```
User
|
v
Streamlit Interface (app.py)
|
v
Image Upload (JPG / JPEG / PNG)
|
v
Resize to 32 x 32 pixels (stated on-screen)
|
v
Trained Model (cifar10_cnn_model.keras)
|
v
Prediction (class + confidence + top-5 probabilities)
|
v
Result Display (Prediction Result / Confidence Level / Top 5 Probabilities / Result Interpretation)
```

9. Streamlit User Interface Design

The following UI components are directly observable in the application screenshots (Figures 2–13):

Component	Observed in Application
Page title with emoji icon	“🧠 CIFAR-10 Image Classifier”

Component	Observed in Application
Subtitle / description text	“Deep Learning Image Prediction using Streamlit” and a one-line usage instruction
Sidebar — “Model Information” panel	Dataset, Model, Framework, Interface, and Input Size fields
Sidebar — “Supported Classes” panel	Bulleted list of the 10 CIFAR-10 classes
Success message (green)	“  Model loaded successfully.”
File uploader widget	“Upload an Image” — accepts JPG, JPEG, or PNG, 200MB per file limit
Informational message (blue)	Prompt to upload an image; note that the image will be resized to 32×32 pixels
Two-column layout	“Uploaded Image” column beside an “Image Information” column
Prediction button	Red-highlighted “  Predict Image” button
Prediction Result section	“Predicted Class” and “Confidence” values shown side-by-side
Confidence Level progress bar	Horizontal bar reflecting the confidence percentage
Top 5 Prediction Probabilities section	Ranked bars for the top five classes with percentage labels
Result Interpretation message (green)	Plain-language sentence summarising the prediction and confidence
Footer text	“Task 6: Creating a Streamlit User Interface L&T EduTech Deep Learning Project”

10. User Input and Widgets

Widget	Accepted Input	How Input Is Used
File uploader (“Upload an Image”)	JPG, JPEG, or PNG image files, up to 200MB per file (as stated on the widget)	The uploaded file is displayed under “Uploaded Image” and its file name, original dimensions, and image mode (e.g., RGBA) are shown under “Image Information.” The application also displays a note that the image will be resized to 32×32 pixels before being passed to the model.
“Predict Image” button	No direct input — triggers processing of the currently uploaded image	Clicking the button runs the model on the uploaded image and populates the “Prediction Result,” “Confidence Level,” “Top 5

Widget	Accepted Input	How Input Is Used
		Prediction Probabilities,” and “Result Interpretation” sections.

11. Model Integration

```
@st.cache_resource
def load_model():

    model_path = os.path.join(
        os.path.dirname(os.path.abspath(__file__)),
        "cifar10_cnn_model.keras"
    )

    if not os.path.exists(model_path):
        raise FileNotFoundError(
            f"Model file not found at: {model_path}"
        )

    return tf.keras.models.load_model(model_path)
```

Key implementation points confirmed from this excerpt:

- Model loading library: TensorFlow/Keras, via `tf.keras.models.load_model()`.
- Model file path: built dynamically using `os.path.join` and `os.path.dirname(os.path.abspath(__file__))`, so the model file (`cifar10_cnn_model.keras`) is located relative to the `app.py` script rather than a hard-coded absolute path.
- Caching: the function is decorated with `@st.cache_resource`, a Streamlit decorator that ensures the model is loaded into memory only once per session rather than on every user interaction, improving performance.
- Error handling: if the model file does not exist at the computed path, a `FileNotFoundError` is explicitly raised with a descriptive message.

12. Image / Data Preprocessing

```
Uploaded Image
|
v
Resize to 32 x 32 pixels
(stated on-screen: "The image will be resized
to 32 x 32 pixels before prediction.")
|
v
Model Input
```

13. Prediction Process

- Predicted class selection: the application displays a single “Predicted Class” value (e.g., DOG, AIRPLANE, HORSE) under “Prediction Result,” consistent with selecting the class of highest probability from the model’s output.

- Confidence calculation: a “Confidence” percentage is shown alongside the predicted class (e.g., 99.99%, 99.95%, 99.98%), and is also visualised as a horizontal “Confidence Level” bar.
- Top predictions: a “Top 5 Prediction Probabilities” section lists the five highest-probability classes in descending order, each with its own percentage and bar.

14. Error Handling

The only error-handling mechanism directly confirmed from the submitted source-code excerpt is the model-loading check shown in Section 11:

- Model file check: before attempting to load the model, the code checks `os.path.exists(model_path)` and raises a `FileNotFoundError` with a descriptive message (“Model file not found at: {model_path}”) if the file is missing.

15. User Interface Screenshots

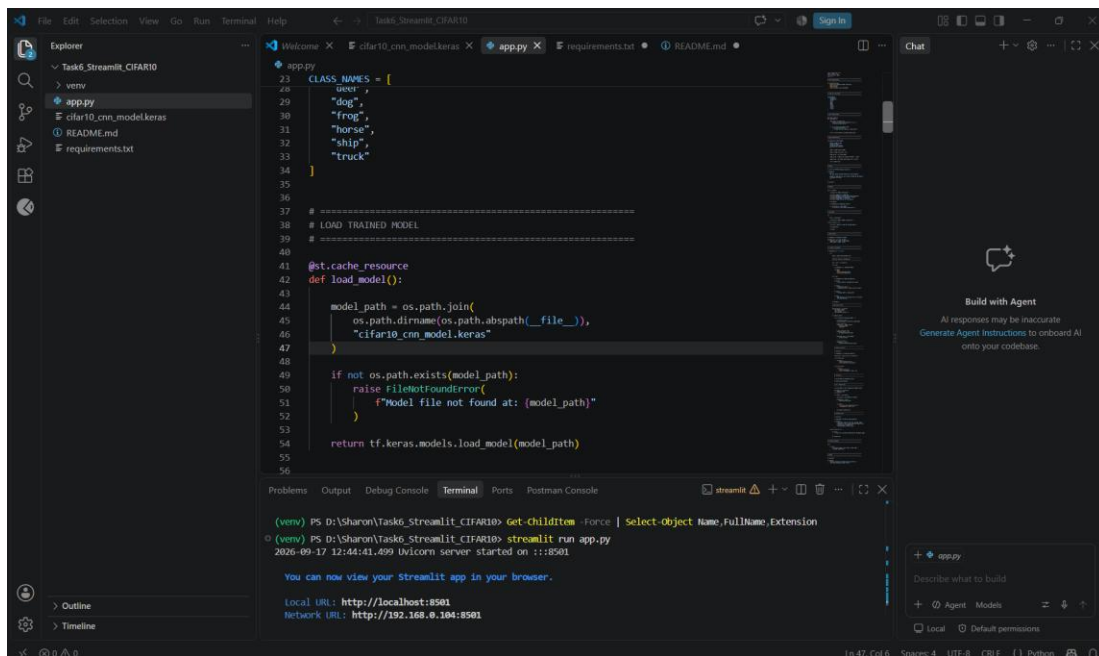


Figure 1: Project Structure and Application Source Code (VS Code)

This screenshot shows the project opened in Visual Studio Code. The Explorer panel (left) lists the project folder Task6_Streamlit_CIFAR10 containing a venv folder, app.py, cifar10_cnn_model.keras, README.md, and requirements.txt. The editor pane (centre) shows part of app.py, including the CLASS_NAMES list and the load_model() function decorated with @st.cache_resource. The integrated terminal (bottom) shows the virtual environment active and the application being started with the command “streamlit run app.py”, after which Streamlit reports a Local URL of http://localhost:8501 and a Network URL of http://192.168.0.104:8501.

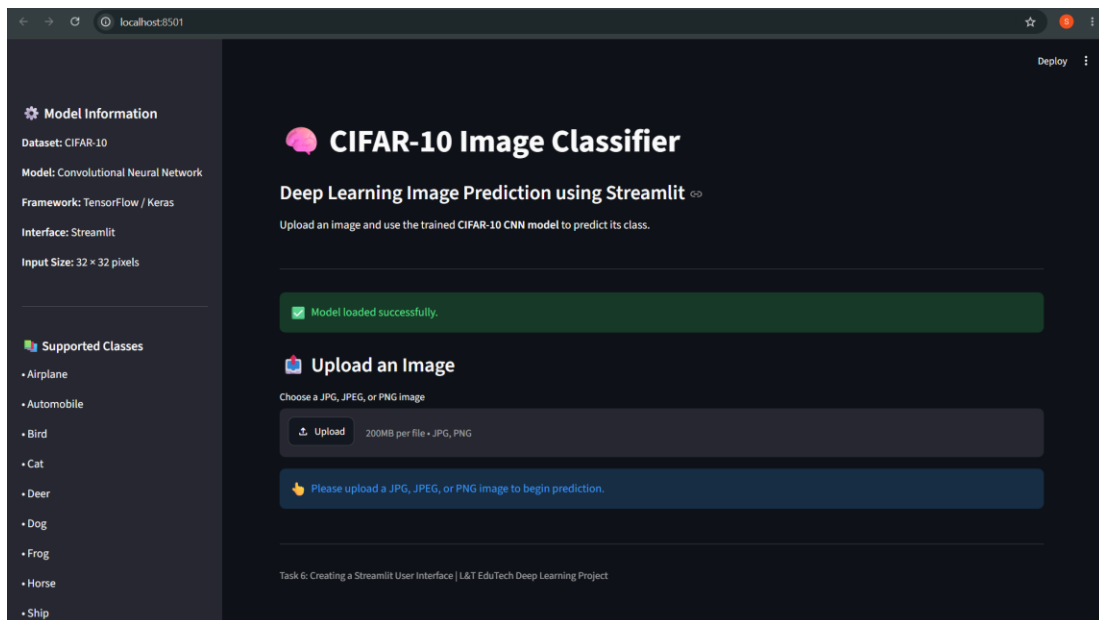


Figure 2: Streamlit Application Home Interface

This is the initial view of the application in the browser at localhost:8501. The sidebar shows the “Model Information” panel (Dataset: CIFAR-10, Model: Convolutional Neural Network, Framework: TensorFlow/Keras, Interface: Streamlit, Input Size: 32×32 pixels) and a “Supported Classes” list of all ten CIFAR-10 classes. The main panel shows the application title “CIFAR-10 Image Classifier,” a success message confirming the model loaded successfully, and the “Upload an Image” file uploader with a prompt to upload a JPG, JPEG, or PNG image to begin prediction.

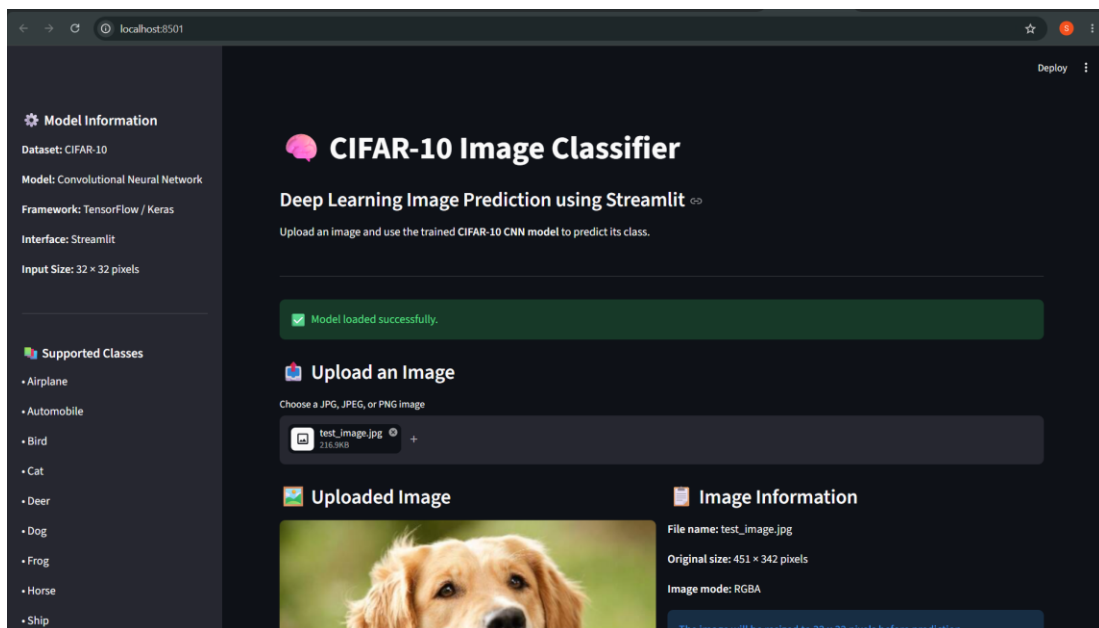


Figure 3: Image Upload Interface — First Test Image (Dog)

A file named test_image.jpg (216.9KB) has been uploaded. The application displays the uploaded image (a dog) under “Uploaded Image” and shows its file name, original size (451×342 pixels), and image mode (RGBA) under “Image Information,” along with a note that the image will be resized to 32×32 pixels before prediction.

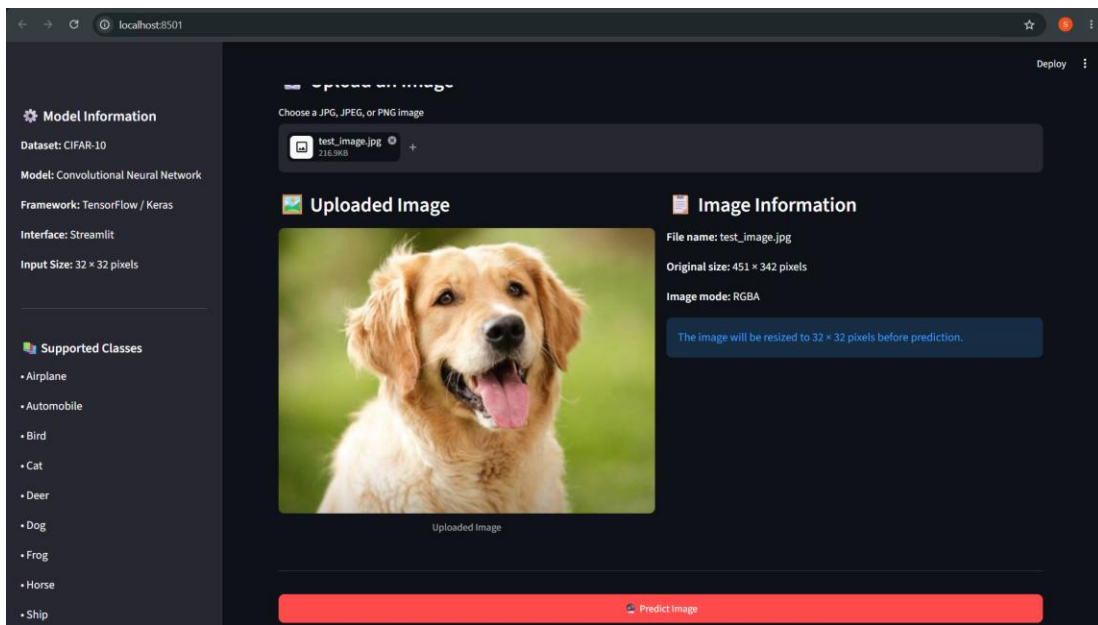


Figure 4: Uploaded Image and Predict Button (Dog)

This view scrolls slightly further down, showing the same uploaded dog image, its Image Information panel, and the red “Predict Image” button that the user clicks to run the model on the uploaded image.

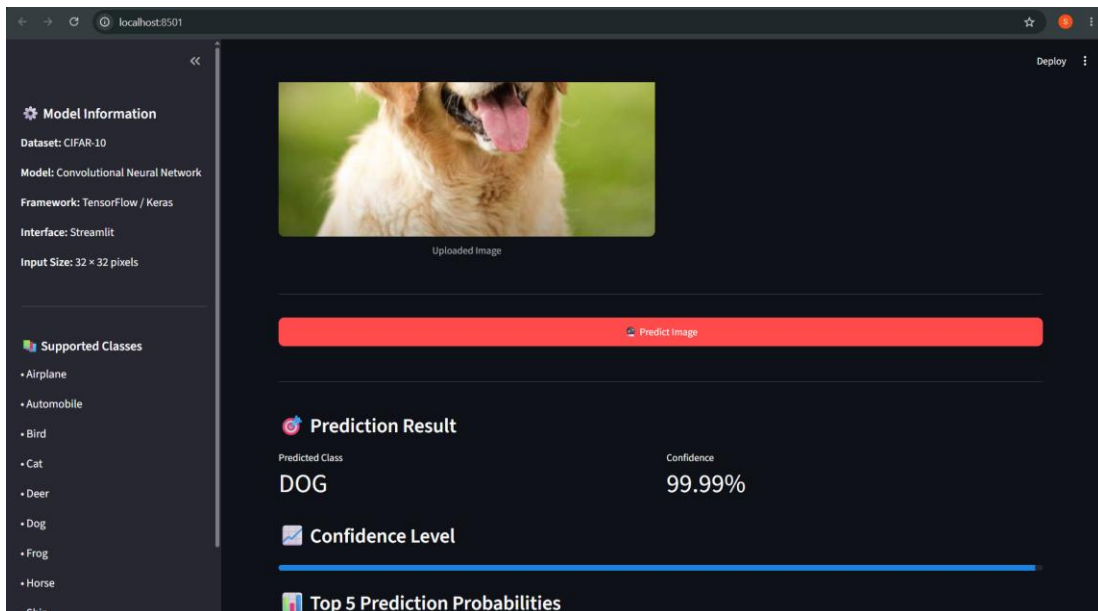


Figure 5: Prediction Result Display (Dog)

After clicking “Predict Image,” the application displays the “Prediction Result” section. The Predicted Class is DOG with a Confidence of 99.99%. A “Confidence Level” bar is shown nearly full, and the beginning of the “Top 5 Prediction Probabilities” section is visible.

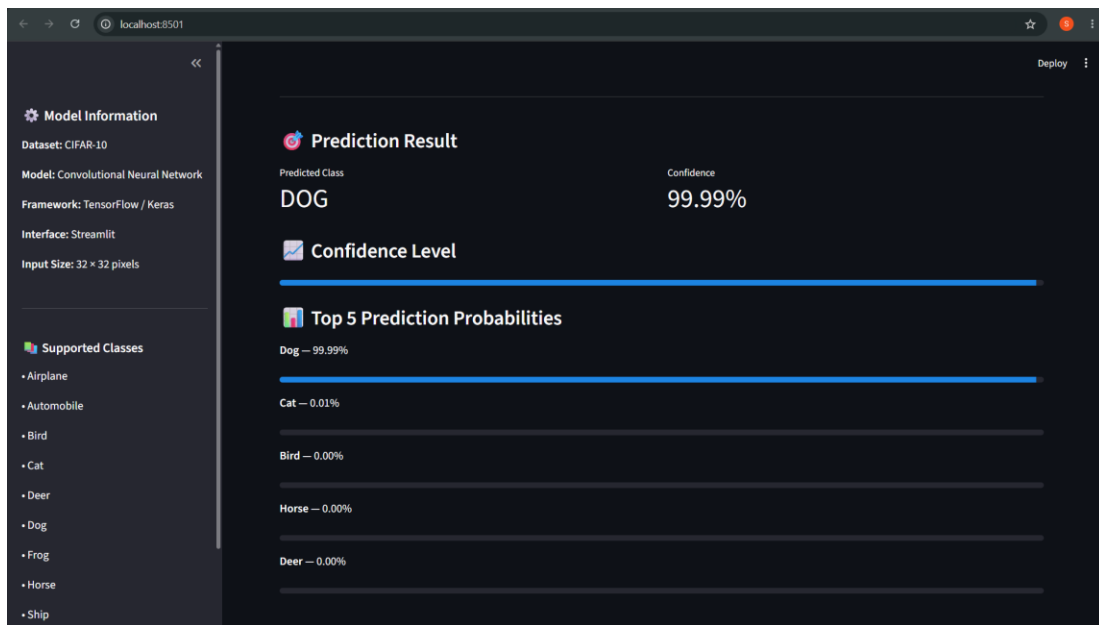


Figure 6: Confidence Level and Top 5 Prediction Probabilities (Dog)

This screenshot shows the full “Top 5 Prediction Probabilities” list for the dog image: Dog — 99.99%, Cat — 0.01%, Bird — 0.00%, Horse — 0.00%, and Deer — 0.00%, each represented with a proportional horizontal bar.

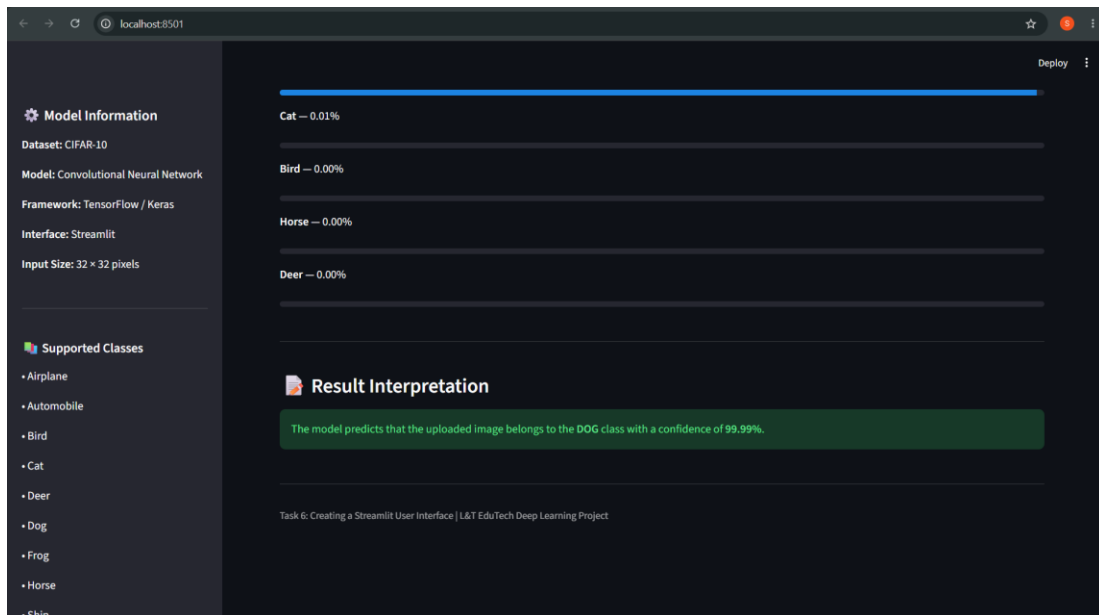


Figure 7: Result Interpretation (Dog)

The remainder of the Top 5 Prediction Probabilities list (Cat, Bird, Horse, Deer) is shown, followed by a green “Result Interpretation” box that states in plain language: “The model predicts that the uploaded image belongs to the DOG class with a confidence of 99.99%.” The application footer — “Task 6: Creating a Streamlit User Interface | L&T EduTech Deep Learning Project” — is visible at the bottom.

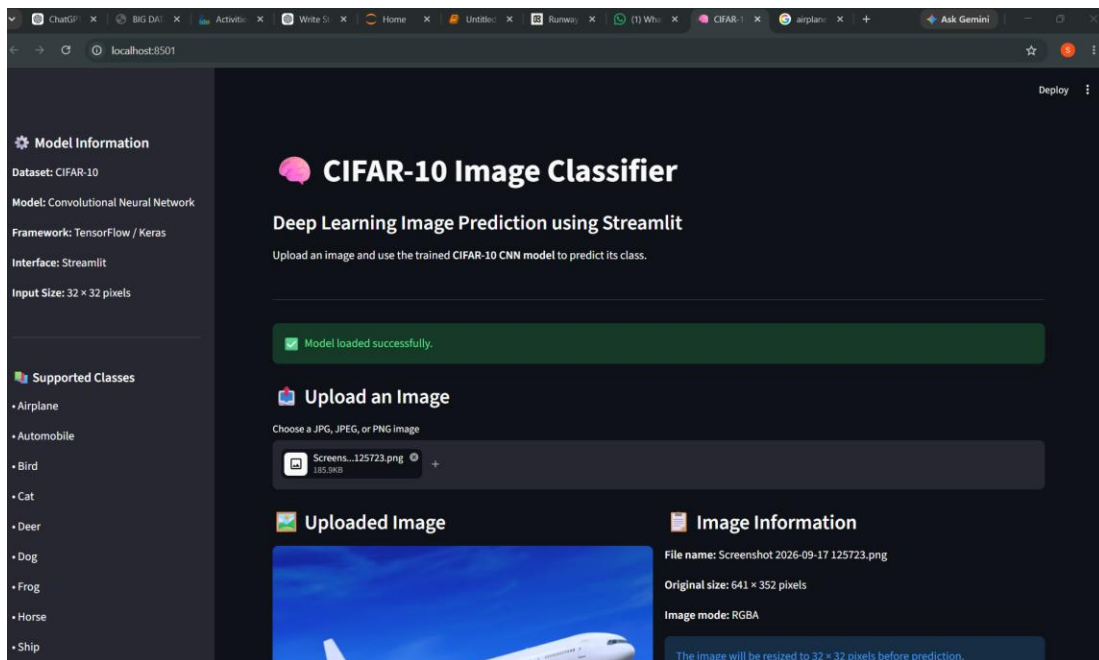


Figure 8: Image Upload Interface — Second Test Image (Airplane)

A second test run is shown, this time with the browser's other open tabs visible at the top. A file named "Screenshot 2026-09-17 125723.png" (185.9KB) has been uploaded, showing an airplane in flight. The Image Information panel reports an original size of 641×352 pixels and image mode RGBA.

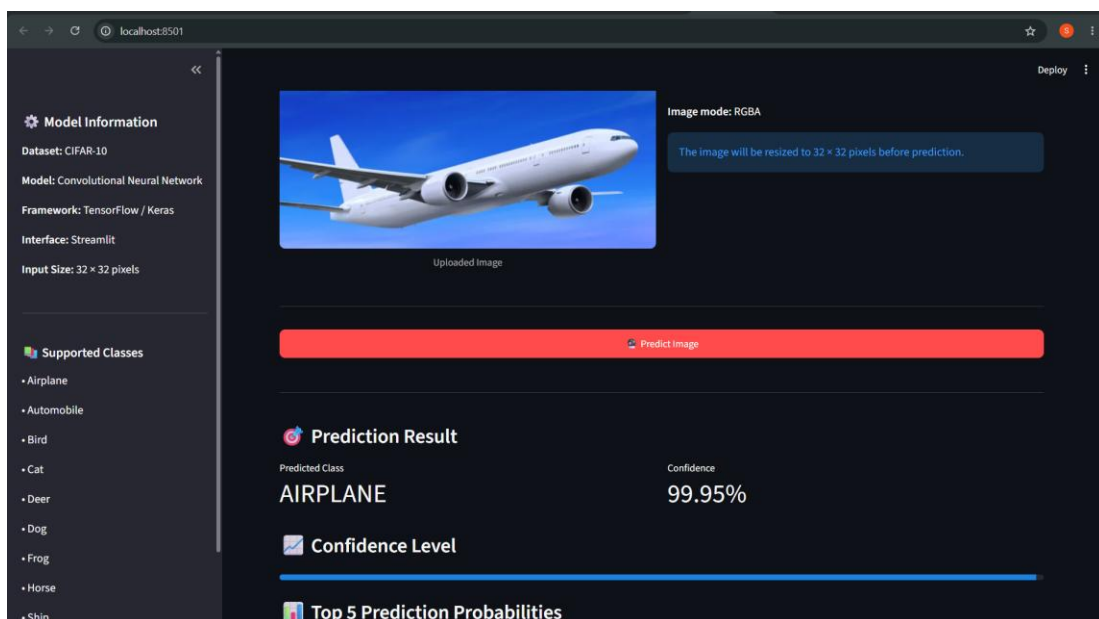


Figure 9: Prediction Result Display (Airplane)

The Predicted Class is AIRPLANE with a Confidence of 99.95%, shown together with the uploaded airplane image, the Predict Image button, and the start of the Top 5 Prediction Probabilities section.

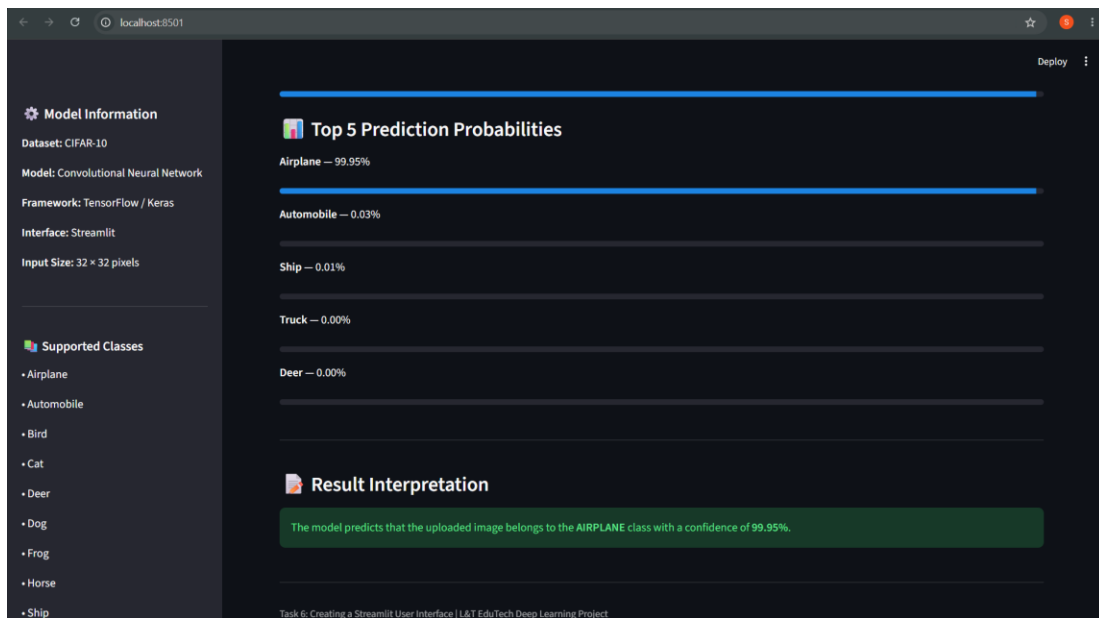


Figure 10: Top 5 Prediction Probabilities and Result Interpretation (Airplane)

The full ranked list for the airplane image is visible: Airplane — 99.95%, Automobile — 0.03%, Ship — 0.01%, Truck — 0.00%, and Deer — 0.00%. The green Result Interpretation box confirms: “The model predicts that the uploaded image belongs to the AIRPLANE class with a confidence of 99.95%.”

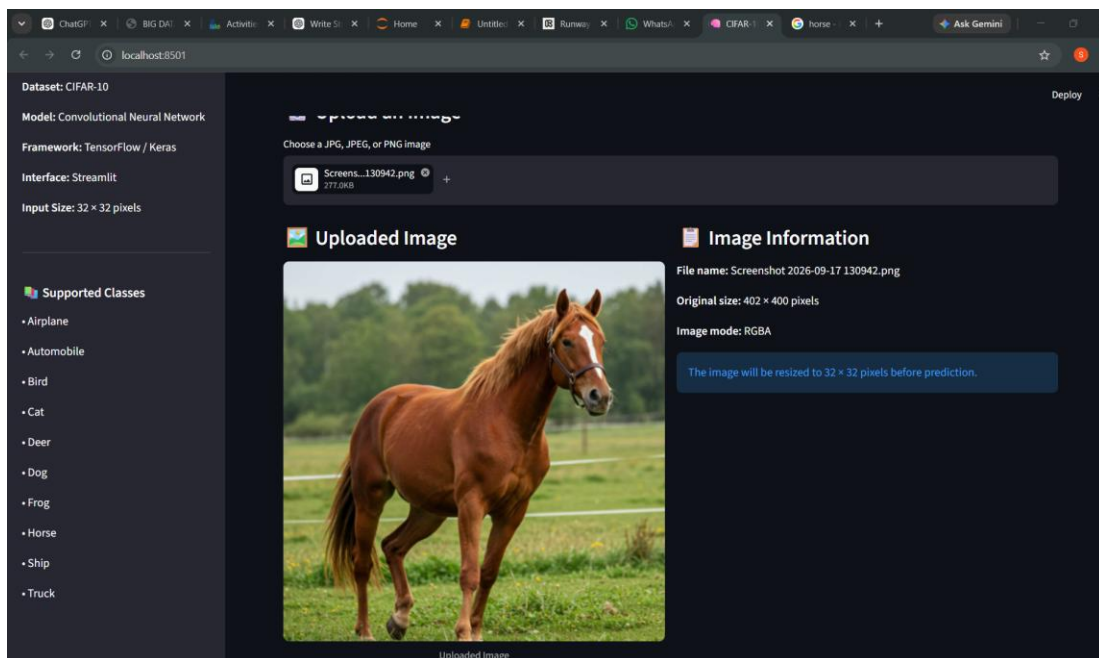


Figure 11: Image Upload Interface — Third Test Image (Horse)

A third test run is shown with a file named “Screenshot 2026-09-17 130942.png” (277.0KB) uploaded, depicting a horse in a grassy field. The Image Information panel reports an original size of 402×400 pixels and image mode RGBA.

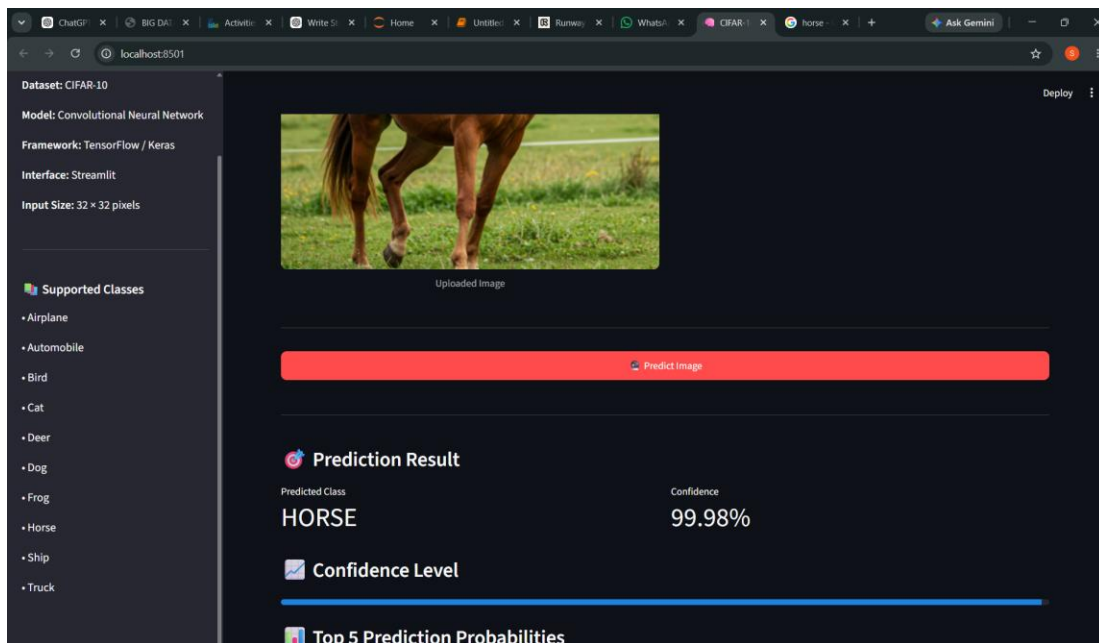


Figure 12: Prediction Result Display (Horse)

The Predicted Class is HORSE with a Confidence of 99.98%, displayed beneath the uploaded horse image and the Predict Image button, with the Confidence Level bar and the start of the Top 5 Prediction Probabilities section visible.

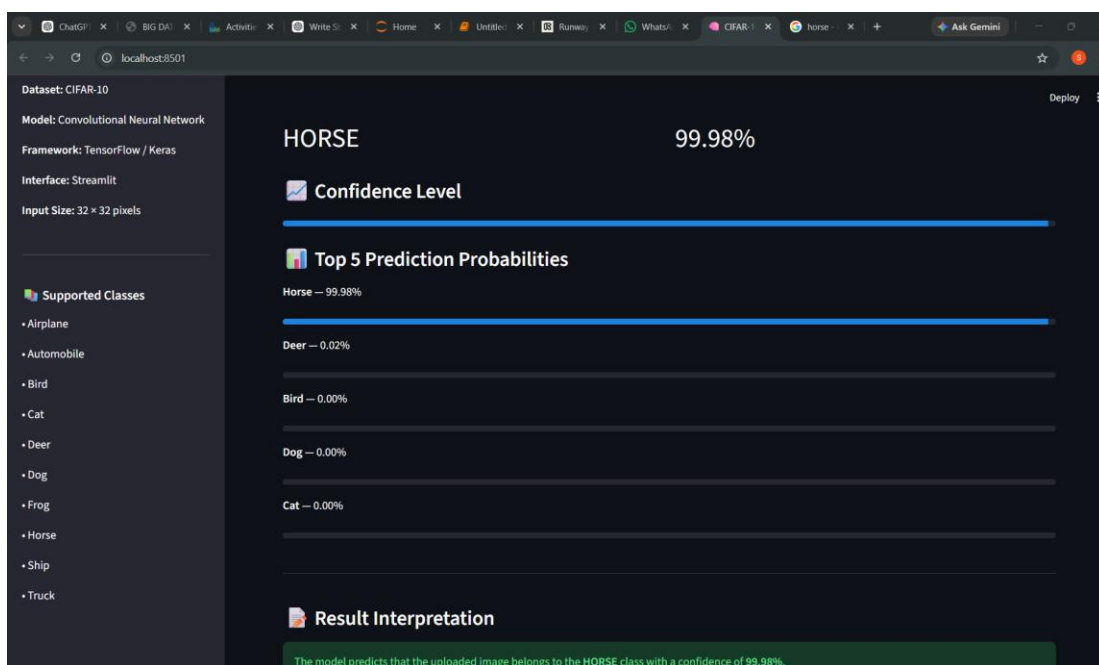


Figure 13: Top 5 Prediction Probabilities and Result Interpretation (Horse)

The full ranked probability list for the horse image is shown: Horse — 99.98%, Deer — 0.02%, Bird — 0.00%, Dog — 0.00%, and Cat — 0.00%. The green Result Interpretation box states: “The model predicts that the uploaded image belongs to the HORSE class with a confidence of 99.98%.”

16. Image Upload Screenshots

The image upload interface was demonstrated across all three test runs. Figures 3, 8, and 11 (presented in Section 15 above) show the upload widget in use with a dog image, an airplane image, and a horse image respectively, each accompanied by the automatically generated Image Information panel (file name, original dimensions, and image mode).

17. Prediction Output Screenshots

Test Image	Predicted Class	Confidence (exact, as shown)
test_image.jpg (dog)	DOG	99.99%
Screenshot 2026-09-17 125723.png (airplane)	AIRPLANE	99.95%
Screenshot 2026-09-17 130942.png (horse)	HORSE	99.98%

18. Prediction Probability Screenshots

The application displays a “Top 5 Prediction Probabilities” breakdown for every prediction. The exact values captured are:

Dog image (Figures 5–6)

Rank	Class	Probability
1	Dog	99.99%
2	Cat	0.01%
3	Bird	0.00%
4	Horse	0.00%
5	Deer	0.00%

Airplane image (Figures 9–10)

Rank	Class	Probability
1	Airplane	99.95%
2	Automobile	0.03%

Rank	Class	Probability
3	Ship	0.01%
4	Truck	0.00%
5	Deer	0.00%

Horse image (Figures 12–13)

Rank	Class	Probability
1	Horse	99.98%
2	Deer	0.02%
3	Bird	0.00%
4	Dog	0.00%
5	Cat	0.00%

19. Results

The application was demonstrated end-to-end on three separate images. The exact, unaltered results were:

Test / Input	Actual Prediction	Confidence	Observation
test_image.jpg — photograph of a dog	DOG	99.99%	Correct prediction; near-certain confidence.
Screenshot 2026-09-17 125723.png — photograph of an airplane	AIRPLANE	99.95%	Correct prediction; near-certain confidence.
Screenshot 2026-09-17 130942.png — photograph of a horse	HORSE	99.98%	Correct prediction; near-certain confidence.

20. Functional Evaluation

Requirement	Evidence in Project	Status
Streamlit UI	Application renders a titled page with sidebar, uploader, and result sections (Figures 2–13)	Implemented
User input widget	File uploader accepting JPG/JPEG/PNG (Figures 2–3, 8, 11)	Implemented

Requirement	Evidence in Project	Status
Model integration	<code>load_model()</code> function using <code>tf.keras.models.load_model</code> with caching and error handling (Figure 1)	Implemented
Prediction output	Predicted Class, Confidence, and Top 5 Probabilities displayed for three separate images (Figures 5–7, 9–10, 12–13)	Implemented
Layout components	Sidebar, two-column layout, progress bars, coloured status messages, footer (Figures 1–13)	Implemented

21. User Experience and Interface Design

The interface uses a two-panel layout, with a persistent sidebar on the left providing model context (dataset, model type, framework, input size, and supported classes) and a main panel on the right dedicated to the upload-and-predict workflow. This separation allows a user to reference model information at any time without it interrupting the primary upload-predict-result flow.

The layout provides immediate feedback at each stage: a green success message confirms the model has loaded before any interaction is required, a blue informational message guides the user to upload an image when none is present, and the uploaded image is displayed alongside its file metadata before prediction is triggered. This reduces the number of steps required to confirm that the correct file was uploaded before committing to a prediction.

The prediction output is presented in three complementary forms — a headline class-and-confidence pair, a horizontal confidence bar, and a ranked top-5 probability breakdown — which allows a user to read the result at a glance (Predicted Class / Confidence) while still being able to inspect the model's full distribution of class probabilities. The application displays a final plain-language “Result Interpretation” sentence, which restates the numeric result in a way that does not require the user to interpret the chart or the raw numbers themselves.

A single “Predict Image” button is used to trigger inference, which keeps the interaction simple and avoids requiring the user to configure additional parameters.

22. Observations

- The interface consistently displays the model's metadata (dataset, model type, framework, input size, supported classes) in the sidebar across every screenshot, regardless of which image was uploaded.
- The model is connected to the interface through a cached loader function (`@st.cache_resource`), which is intended to load the .keras model file once and reuse it across interactions rather than reloading it on every prediction.
- Uploaded inputs are handled through a single file-uploader widget restricted to JPG, JPEG, and PNG formats, and the application immediately reports the uploaded file's original dimensions and colour mode before any resizing occurs.

- Predictions are displayed using a consistent three-part layout (headline result, confidence bar, top-5 ranked list) across all three demonstrated test cases, indicating a uniform result-rendering design regardless of the input image.
- In all three demonstrated cases, the top predicted class carried a confidence above 99.9%, with the remaining four classes in the top-5 list each at or below 0.03%, indicating the model produced sharply peaked (highly confident) probability distributions for these particular test images.
- The layout affects usability positively in that no scrolling is required to view the uploaded image or trigger a prediction on a standard desktop browser window, based on the screenshots provided; some scrolling is required to view the full Top 5 Probabilities and Result Interpretation sections on a single screen.

23. Limitations

- Local-only execution: the application was demonstrated only via a Local URL (<http://localhost:8501>) and a Network URL restricted to a local network address (<http://192.168.0.104:8501>); no public/online deployment evidence was submitted.
- No authentication: no login, access control, or user-management mechanism is visible anywhere in the interface.
- Model-dependent prediction accuracy: only three test images were demonstrated, all of which produced correct, high-confidence predictions; broader accuracy across a larger or more varied test set is not evidenced in the submitted materials.
- Limited supported formats: the file uploader accepts only JPG, JPEG, and PNG images, as explicitly stated on the widget.
- Incomplete source-code visibility: only a partial excerpt of app.py (the CLASS_NAMES list and load_model function) was captured, so the full implementation — including preprocessing and prediction logic — cannot be fully documented or verified in this report.
- No requirements.txt contents available: exact dependency versions could not be confirmed.

24. Future Enhancements

The following are suggested as potential future enhancements. They are not part of the current implementation and are not claimed to exist in the submitted project.

- Online deployment (e.g., via Streamlit Community Cloud or another hosting service) to make the application accessible outside the local network.
- Additional UI controls, such as a confidence threshold slider or a toggle between top-5 and full 10-class probability views.
- A prediction history panel to let users review previously uploaded images and results within the same session.
- Enhanced visualization of model confidence, such as a bar chart comparing all 10 class probabilities rather than only the top 5.
- Support for predicting on multiple uploaded images in a single batch.
- Improved accessibility features, such as screen-reader-friendly labels for the uploader and result sections.

- Support for additional trained models (e.g., allowing the user to select between multiple .keras models from a dropdown).

25. Conclusion

The submitted Task 6 project implements a Streamlit-based web interface, “CIFAR-10 Image Classifier,” that connects a previously trained CNN model (cifar10_cnn_model.keras) to a simple upload-and-predict workflow. Based strictly on the verified evidence — the visible source code for model loading, the sidebar's stated model metadata, and three complete end-to-end demonstrations — the application successfully allows a user to upload an image, trigger a prediction with a single button, and view the predicted class together with a confidence score and a ranked breakdown of the top five class probabilities. This verified functionality directly addresses the task's stated objective of building a user-friendly web interface for interacting with a deep learning prediction model. Aspects of the implementation that could not be verified from the submitted materials — such as the exact preprocessing and prediction code, the full requirements.txt contents, and broader testing beyond the three demonstrated images — have been explicitly identified as such rather than assumed.

26. References

- Streamlit Inc. Streamlit Documentation. <https://docs.streamlit.io>
- TensorFlow. TensorFlow Keras API Documentation. https://www.tensorflow.org/api_docs/python/tf/keras
- Python Software Foundation. Python 3 Documentation. <https://docs.python.org/3/>
- Krizhevsky, A. CIFAR-10 Dataset. <https://www.cs.toronto.edu/~kriz/cifar.html>

27. Appendix

Appendix A — Source Code (app.py)

```
. import streamlit as st
import tensorflow as tf
import numpy as np
from PIL import Image
import os

# =====
# PAGE CONFIGURATION
# =====

st.set_page_config(
    page_title="CIFAR-10 Image Classifier",
    page_icon="🐶",
    layout="wide",
    initial_sidebar_state="expanded"
```

```

)

# =====
# CIFAR-10 CLASS NAMES
# =====

CLASS_NAMES = [
    "airplane",
    "automobile",
    "bird",
    "cat",
    "deer",
    "dog",
    "frog",
    "horse",
    "ship",
    "truck"
]

# =====
# LOAD TRAINED MODEL
# =====

@st.cache_resource
def load_model():

    model_path = os.path.join(
        os.path.dirname(os.path.abspath(__file__)),
        "cifar10_cnn_model.keras"
    )

    if not os.path.exists(model_path):
        raise FileNotFoundError(
            f"Model file not found at: {model_path}"
        )

    return tf.keras.models.load_model(model_path)

# =====
# IMAGE PREPROCESSING
# =====

def preprocess_image(image):
    """
    Convert image to RGB,

```

```

    resize to 32x32,
    normalize pixel values,
    and add batch dimension.
    """

    image = image.convert("RGB")

    image = image.resize((32, 32))

    image_array = np.array(image)

    image_array = image_array.astype("float32") / 255.0

    image_array = np.expand_dims(image_array, axis=0)

    return image_array

# =====
# HEADER
# =====

st.title("🤖 CIFAR-10 Image Classifier")

st.markdown(
    """
    ### Deep Learning Image Prediction using Streamlit

    Upload an image and use the trained **CIFAR-10 CNN model**
    to predict its class.
    """
)

st.divider()

# =====
# SIDEBAR
# =====

with st.sidebar:

    st.header("⚙️ Model Information")

    st.write("**Dataset:** CIFAR-10")
    st.write("**Model:** Convolutional Neural Network")
    st.write("**Framework:** TensorFlow / Keras")
    st.write("**Interface:** Streamlit")

```

```

st.write("**Input Size:** 32 x 32 pixels")

st.divider()

st.subheader("📁 Supported Classes")

for class_name in CLASS_NAMES:
    st.write(f"• {class_name.capitalize()}")

# =====
# LOAD MODEL
# =====

try:

    model = load_model()

    st.success("✅ Model loaded successfully.")

except Exception as e:

    st.error("❌ Unable to load the trained model.")

    st.exception(e)

    st.stop()

# =====
# IMAGE UPLOAD
# =====

st.subheader("📁 Upload an Image")

uploaded_file = st.file_uploader(
    "Choose a JPG, JPEG, or PNG image",
    type=["jpg", "jpeg", "png"]
)

# =====
# IF IMAGE IS UPLOADED
# =====

if uploaded_file is not None:

    try:

```

```

image = Image.open(uploaded_file)

# -----
# DISPLAY IMAGE AND INFORMATION
# -----

col1, col2 = st.columns(2)

with col1:

    st.subheader("🖼️ Uploaded Image")

    st.image(
        image,
        caption="Uploaded Image",
        use_container_width=True
    )

with col2:

    st.subheader("📄 Image Information")

    st.write(
        f"""File name:** {uploaded_file.name}"""
    )

    st.write(
        f"""Original size:** "
        f"{image.size[0]} × {image.size[1]} pixels"
    )

    st.write(
        f"""Image mode:** {image.mode}"""
    )

    st.info(
        "The image will be resized to 32 × 32 pixels "
        "before prediction."
    )

st.divider()

# -----
# PREDICTION BUTTON
# -----

predict_button = st.button(

```

```

    "🖼️ Predict Image",
    type="primary",
    use_container_width=True
)

if predict_button:

    with st.spinner("Analyzing image..."):

        # Preprocess image
        processed_image = preprocess_image(image)

        # Make prediction
        predictions = model.predict(
            processed_image,
            verbose=0
        )

        # Find predicted class
        predicted_index = int(
            np.argmax(predictions[0])
        )

        predicted_class = CLASS_NAMES[
            predicted_index
        ]

        # Get confidence
        confidence = float(
            predictions[0][predicted_index]
        )

        # -----
        # PREDICTION RESULT
        # -----

    st.divider()

    st.subheader("🔮 Prediction Result")

    result_col1, result_col2 = st.columns(2)

    with result_col1:

        st.metric(
            label="Predicted Class",
            value=predicted_class.upper()
        )

```

```

with result_col2:

    st.metric(
        label="Confidence",
        value=f"{confidence * 100:.2f}%"
    )

    # -----
    # CONFIDENCE
    # -----

    st.write("### 📊 Confidence Level")

    st.progress(confidence)

    # -----
    # TOP 5 PREDICTIONS
    # -----

    st.write("### 📊 Top 5 Prediction Probabilities")

    top_indices = np.argsort(
        predictions[0]
    )[::-1][:5]

    for index in top_indices:

        class_name = CLASS_NAMES[int(index)]

        probability = float(
            predictions[0][index]
        )

        st.write(
            f"**{class_name.capitalize()}** - "
            f"{probability * 100:.2f}%"
        )

        st.progress(probability)

    # -----
    # INTERPRETATION
    # -----

    st.divider()

    st.subheader("📄 Result Interpretation")

```

```

        st.success(
            f"The model predicts that the uploaded image "
            f"belongs to the **{predicted_class.upper()}** "
            f"class with a confidence of "
            f"**{confidence * 100:.2f}%**."
        )

    except Exception as e:

        st.error(
            "✖ There was a problem processing the uploaded image."
        )

        st.exception(e)

# =====
# NO IMAGE UPLOADED
# =====

else:

    st.info(
        "👉 Please upload a JPG, JPEG, or PNG image "
        "to begin prediction."
    )

# =====
# FOOTER
# =====

st.divider()

st.caption(
    "Task 6: Creating a Streamlit User Interface | "
    "L&T EduTech Deep Learning Project"
)

```

Appendix B — Requirements File

```
streamlit==1.64.0
tensorflow==2.21.0
numpy
pillow
```

Appendix C — Additional Screenshots

Deployed Streamlit app

<https://sharonvarghesethoppil-l-t-task-6-app-bc6t1c.streamlit.app/>

